

Трейт в Rust як алгебра поведінки типів

СТРУКТУРА

Data / State



ТРЕЙТ

Behavior / Logic

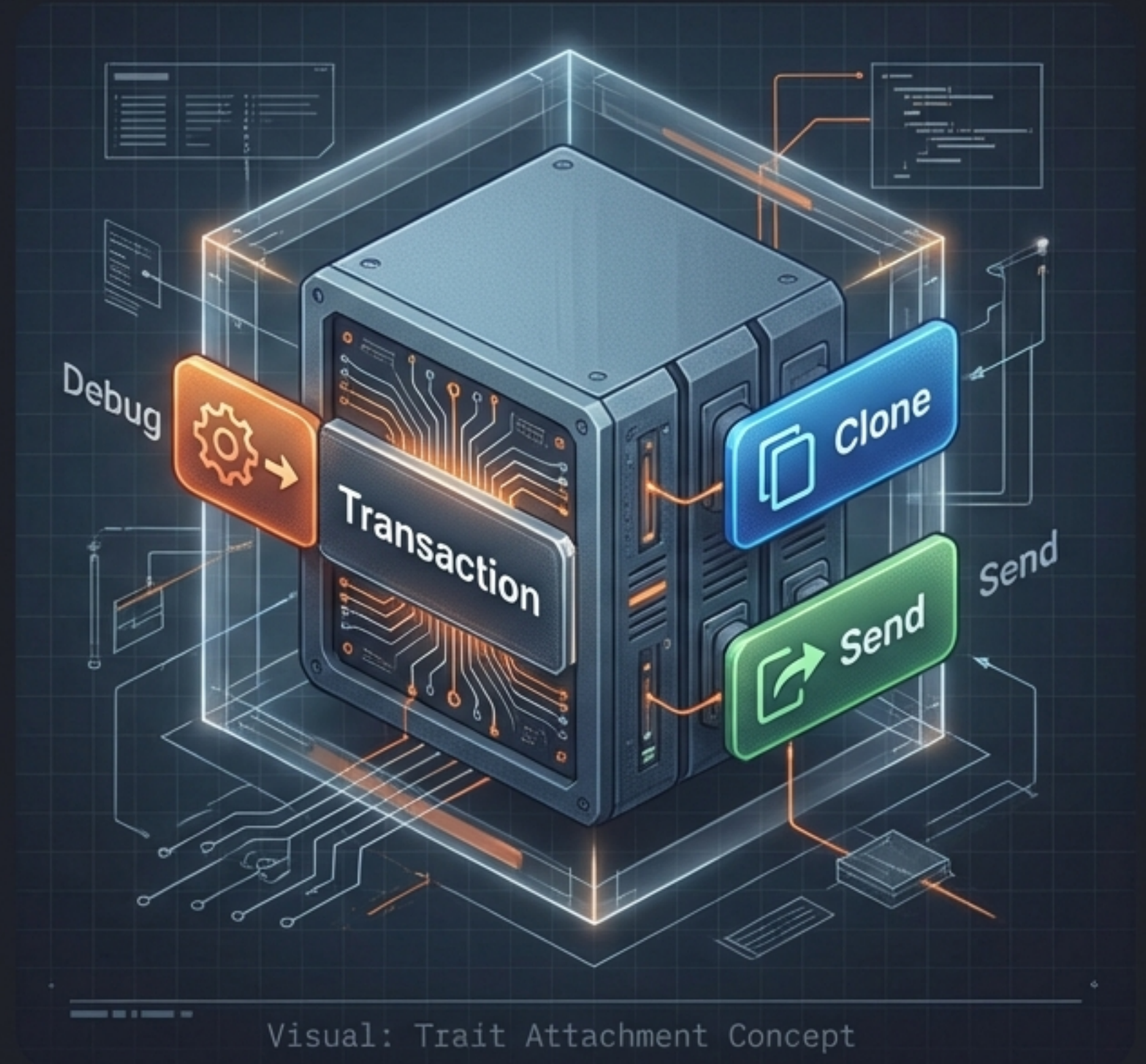


Трейт $\neq$ Інтерфейс

- Трейт — це формалізація властивостей типів.
- Поведінка не успадковується, а приєднується.
- Математичний опис можливостей.

Трейти як концепція: більше, ніж інтерфейс

- Контракт поведінки приєднується до типу.
- Властивості ортогональні до суті даних.
- Трейт визначає допустимі операції.
- Zero-cost abstraction (Нульова вартість).



Як трейти зменшують coupling

```
// Функція залежить від властивості, а не від типу  
fn sort<T: Ord>(items: &mut [T]) {  
    // Implementation  
}
```

Математична властивість

- Функція не знає конкретного класу.
- Алгоритм вимагає лише можливості впорядкування.
- Рішення про реалізацію відкладено до компіляції.

Як трейти підвищують композиційність

- Поведінка ортогональна до структури.
- Вимоги формулюються як властивості.
- Зниження зв'язності (Low Coupling).

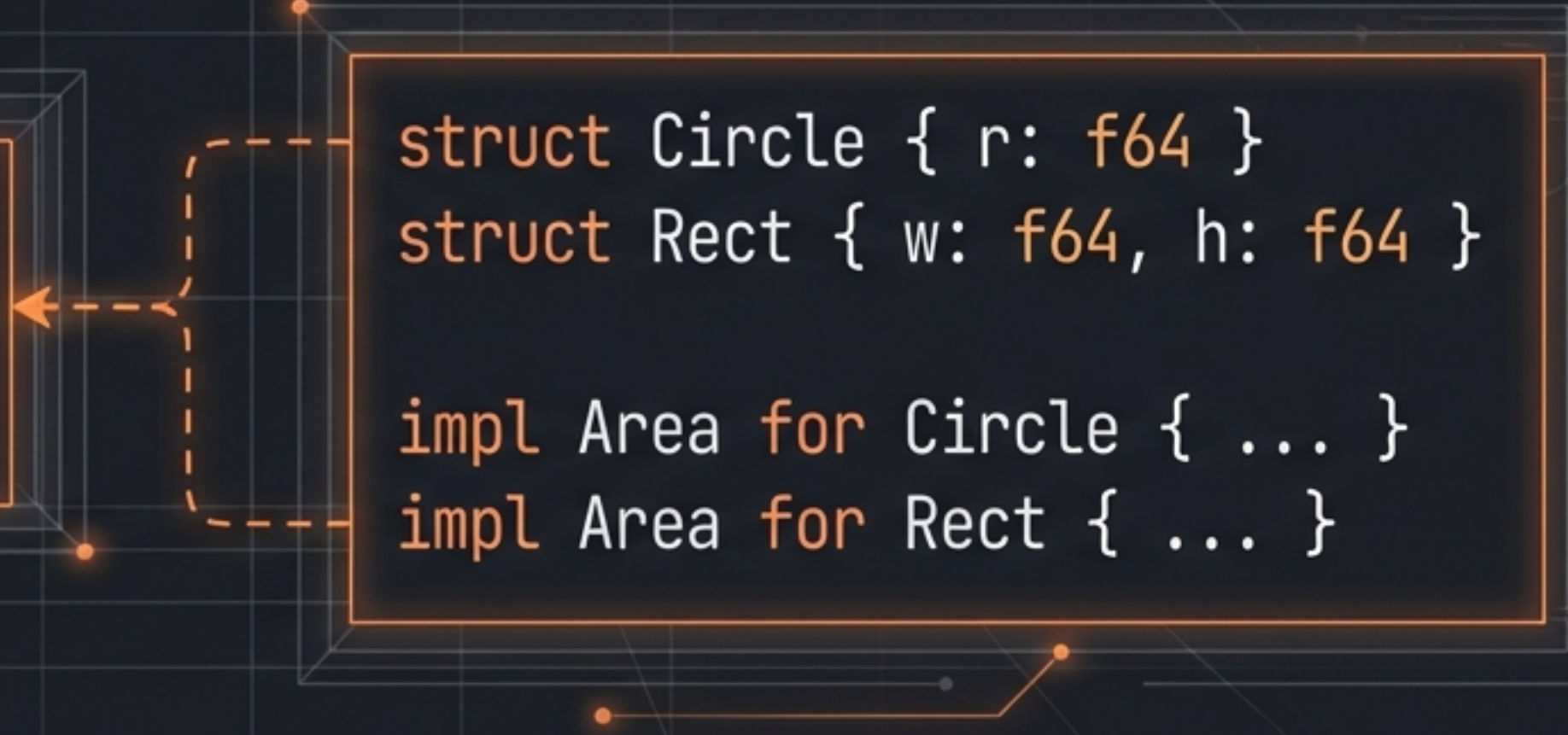
```
fn inspect<T: Debug>(item: T) {  
    println("{:?}", item);  
}
```



Абстрагування поведінки (Behavioral Abstraction)

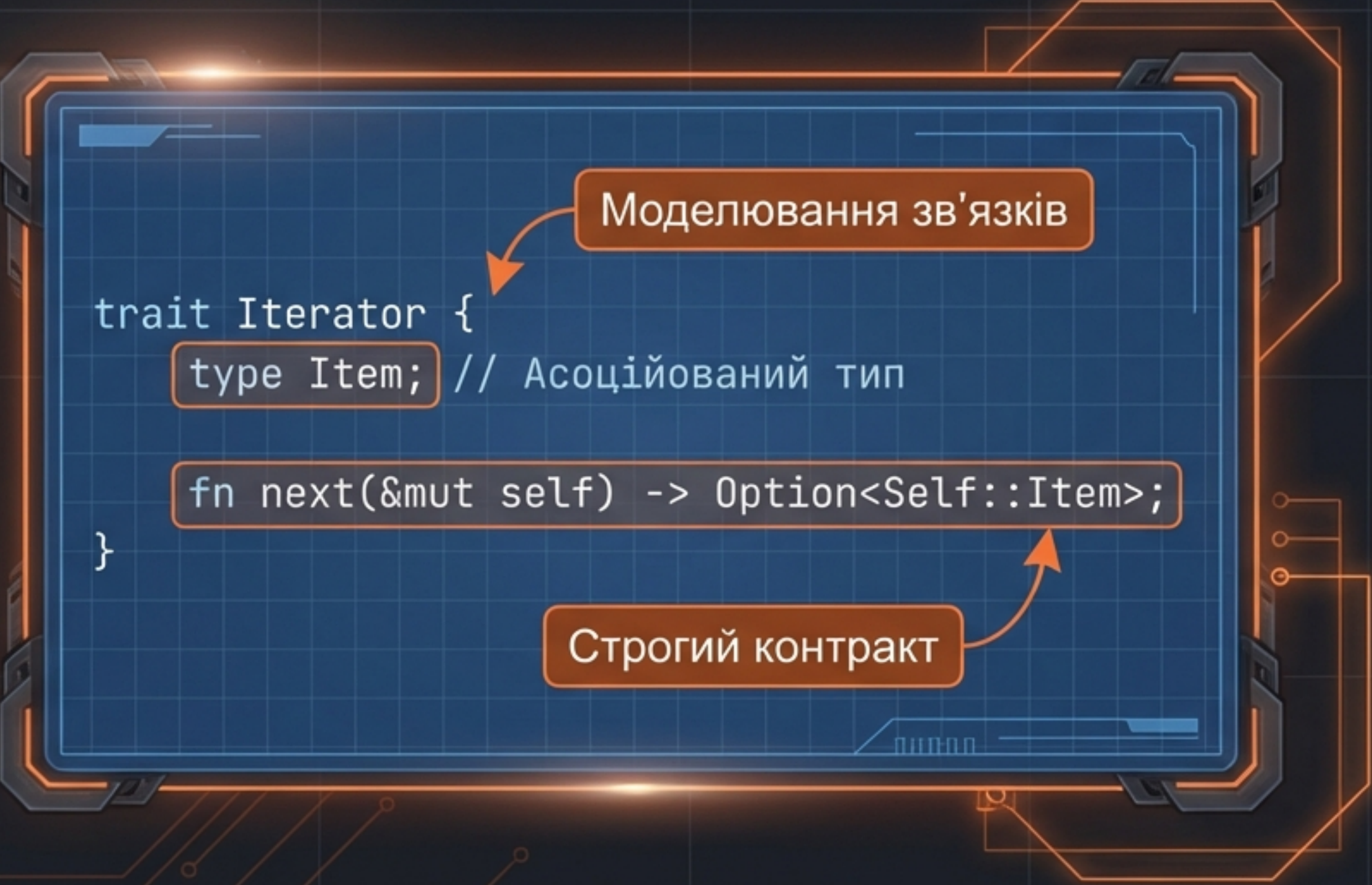
```
trait Area {  
    fn area(&self) -> f64;  
}
```

```
struct Circle { r: f64 }  
struct Rect { w: f64, h: f64 }  
  
impl Area for Circle { ... }  
impl Area for Rect { ... }
```



Декомпозиція: Circle та Rect не мають спільного предка, але мають спільну поведінку.

Контракти інтерфейсного типу



The diagram shows a Rust trait definition for `Iterator` inside a blue-bordered box with a grid background. Two orange callout boxes with arrows point to specific parts of the code: one points to `type Item;` and the other points to `fn next(&mut self) -> Option<Self::Item>;`. The entire box is surrounded by orange circuit-like lines.

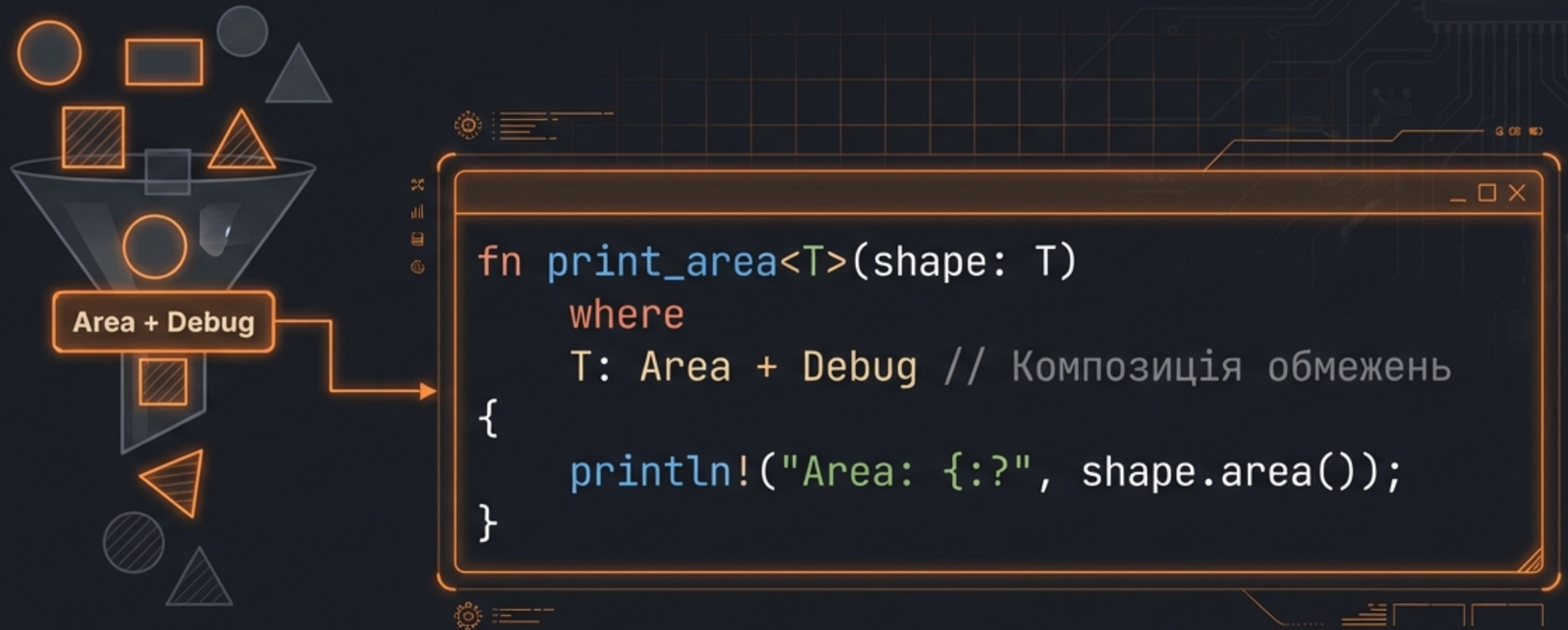
```
trait Iterator {  
    type Item; // Асоційований тип  
  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

Моделювання зв'язків

Строгий контракт

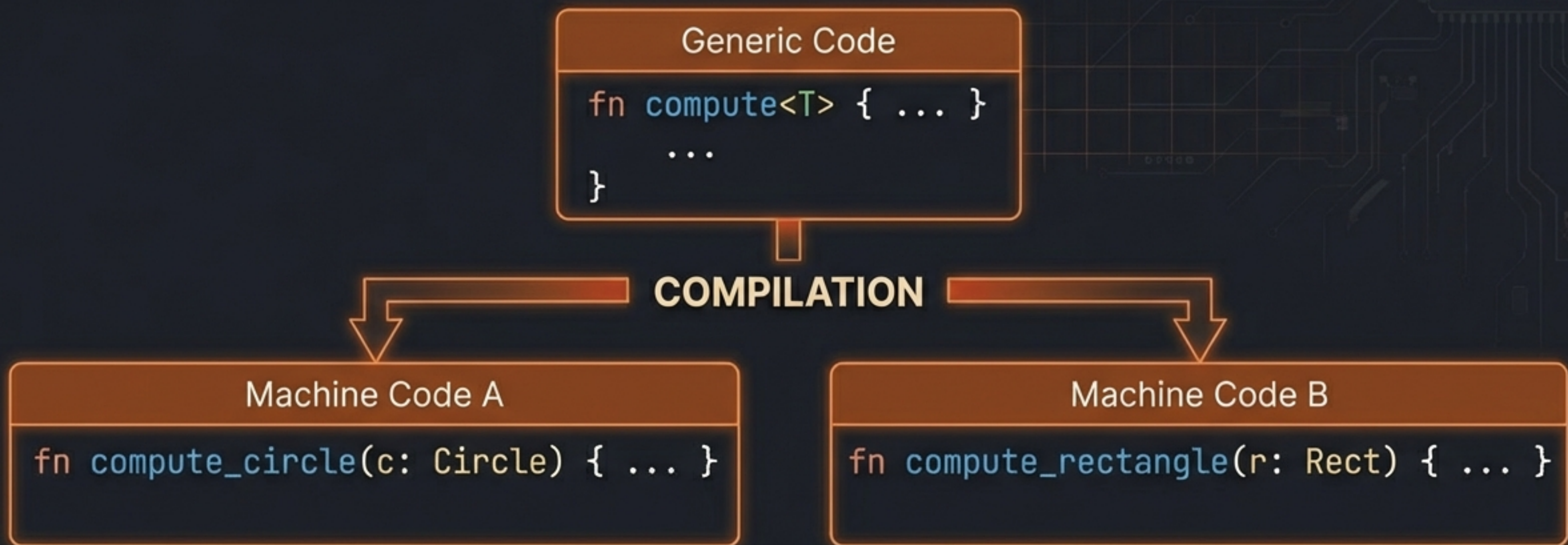
- Перевірка компілятором.
- Гарантія виконання зобов'язань.
- Більше ніж просто методи.

Поліморфізм на основі обмежень (Ad-hoc)



- Узагальнення для будь-яких типів, що задовольняють контракт.
- Синтаксис ``where`` для читабельності.

Статичне розв'язання: Мономорфізація



- Окремий код для кожного типу.
- Відсутність vtable.
- Максимальна оптимізація (Inlining).

Параметричний поліморфізм з обмеженнями

```
fn max<T: Ord>(a: T, b: T) -> T {  
    if a > b { a } else { b }  
}
```

Вимога для порівняння

- Алгоритм вимагає конкретних властивостей.
- Без Ord операція > неможлива.
- Формалізація вимог логіки.

Реалізації поза типом (Orphan Rule)

Local Crate

```
impl MyTrait for i32 {  
    ...  
}
```



External Crate

```
impl ForeignTrait for ForeignType {  
    ...  
}
```

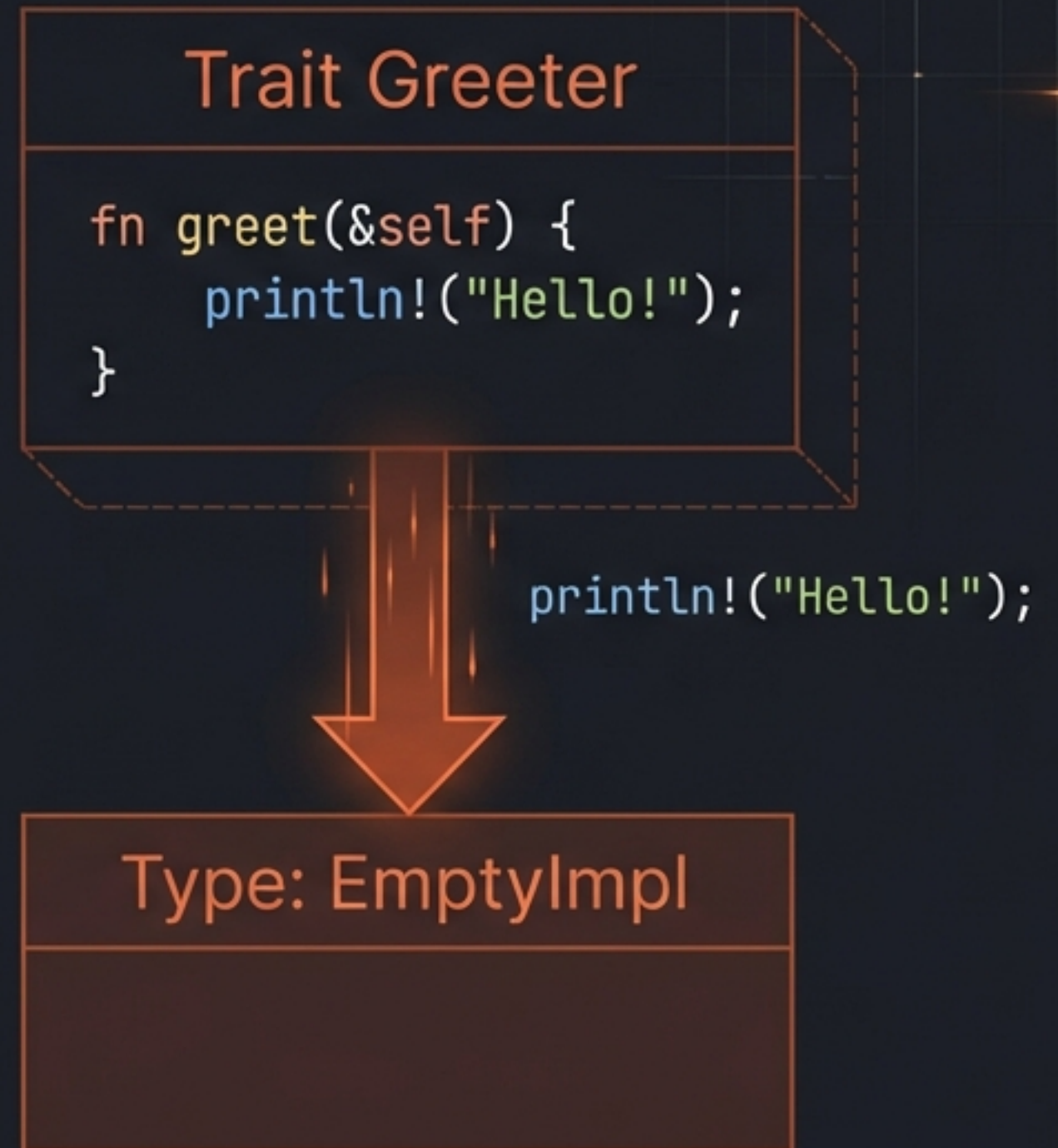


- Правило: Трейт або Тип має бути локальним.
- Мета: Когерентність (уникнення конфліктів).
- Рішення: Патерн `NewType`.

Default-реалізації методів

```
trait Greeter {  
    // Стандартна реалізація  
    fn greet(&self) {  
        println!("Hello!");  
    }  
}
```

- Стандартна поведінка 'з коробки'.
- Зменшення дублювання (boilerplate).
- Можливість перевизначення (Override).



Extension Mechanism: Розширення чужих типів



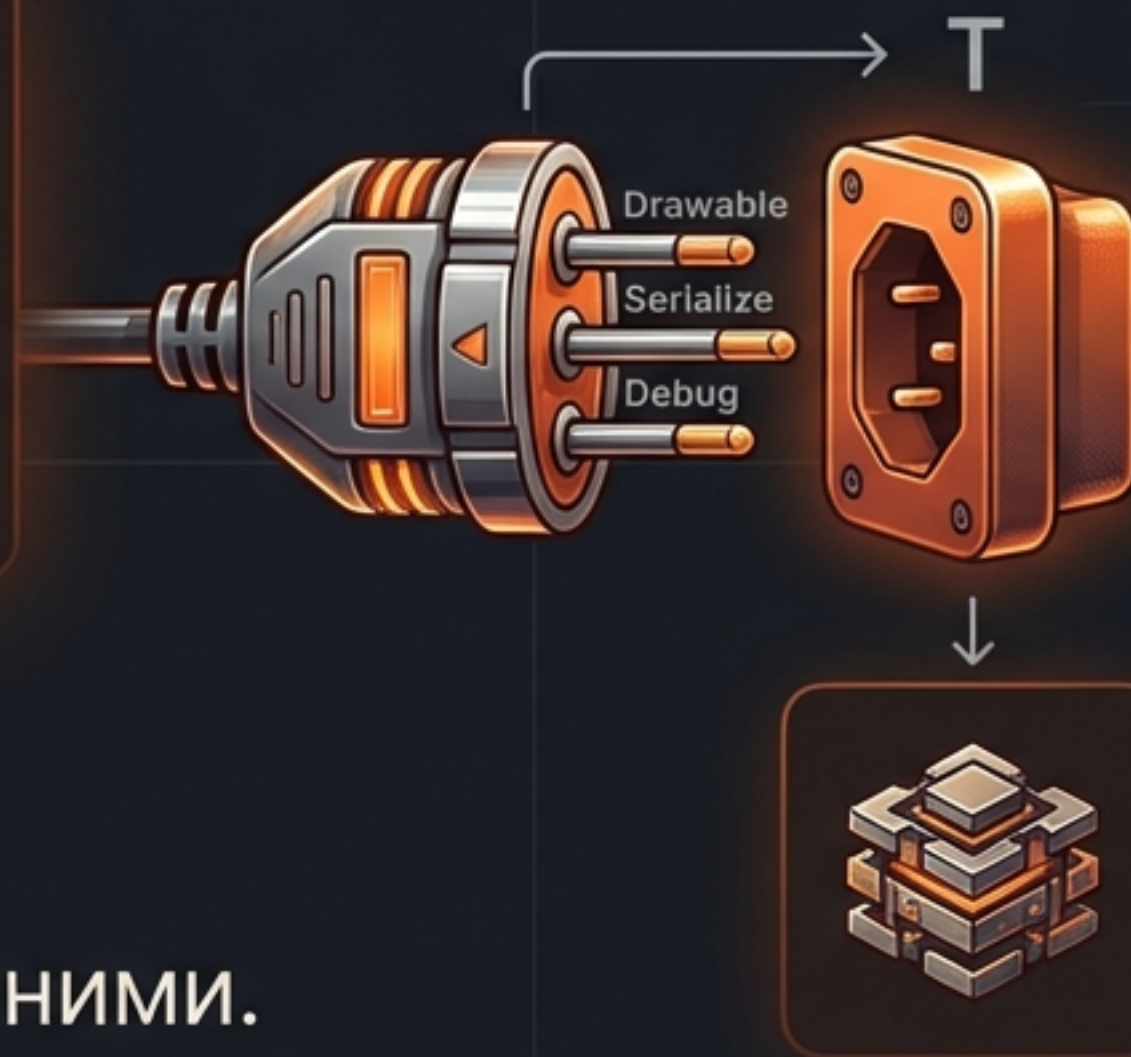
- Додавання методів до примітивів.
- Альтернатива наслідуванню.
- Без конфліктів завдяки Orphan Rule.

```
trait IsEven {  
    fn is_even(&self) -> bool;  
}  
  
impl IsEven for i32 {  
    fn is_even(&self) -> bool { self % 2 == 0 }  
}  
  
// 10.is_even()
```


Композиція поведінки (Trait Composition)

```
fn render<T>(object: T)
where
    T: Drawable + Serialize + Debug
{ ... }
```

- Об'єднання контрактів.
- Тип “володіє” властивостями, а не “є” ними.
- Жодних проблем “діамантового спадкування”.



Inter

Динамічна диспетчеризація (dyn Trait)

JetBrains Mono

```
let animals: Vec<Box<dyn Animal>> = vec![  
    Box::new(Dog {}),  
    Box::new(Cat {}),  
];
```

STACK

Vec

Box

Box

...

...

HEAP

Dog data

vtable
(Virtual Table)

Dog::make_sound

Dog::make_sound

...

...

Cat data

Cat::make_sound

...

...

- Гнучкість ціною продуктивності.
- Гетерогенні колекції (різні типи разом).
- Runtime overhead (vtable lookup).

Статика vs Динаміка

Static Dispatch (Generics)



Generics: Monomorphization

Compile-time Resolution

- Pros: Zero Cost
- Pros: Inlining
- Cons: Code Size (Binary bloat)

Dynamic Dispatch (dyn Trait)



dyn Trait: vtable Lookup

Runtime Resolution

- Pros: Heterogeneous types
- Pros: Smaller Binary
- Cons: Slower
- Cons: No Inlining

У Rust вибір завжди явний.

Обмеження на рівні типів: Marker Traits

JetBrains Mono

```
// Гарантія безпеки потоків  
fn spawn<T: Send + Sync>(task: T) { ... }
```

Безпечне переміщення даних між потоками.



Belt B (Thread B)

- Трейт як логічний предикат.
- `Send` / `Sync`: Гарантії конкурентності.
- Компілятор перевіряє інваріанти.

Автоматичне виведення: `#[derive]`

JetBrains Mono

```
#[derive(Debug, PartialEq, Clone)]  
struct Point {  
    x: i32,  
    y: i32,  
}
```

Macro

Input: `struct Point`

`#[derive(...)]`

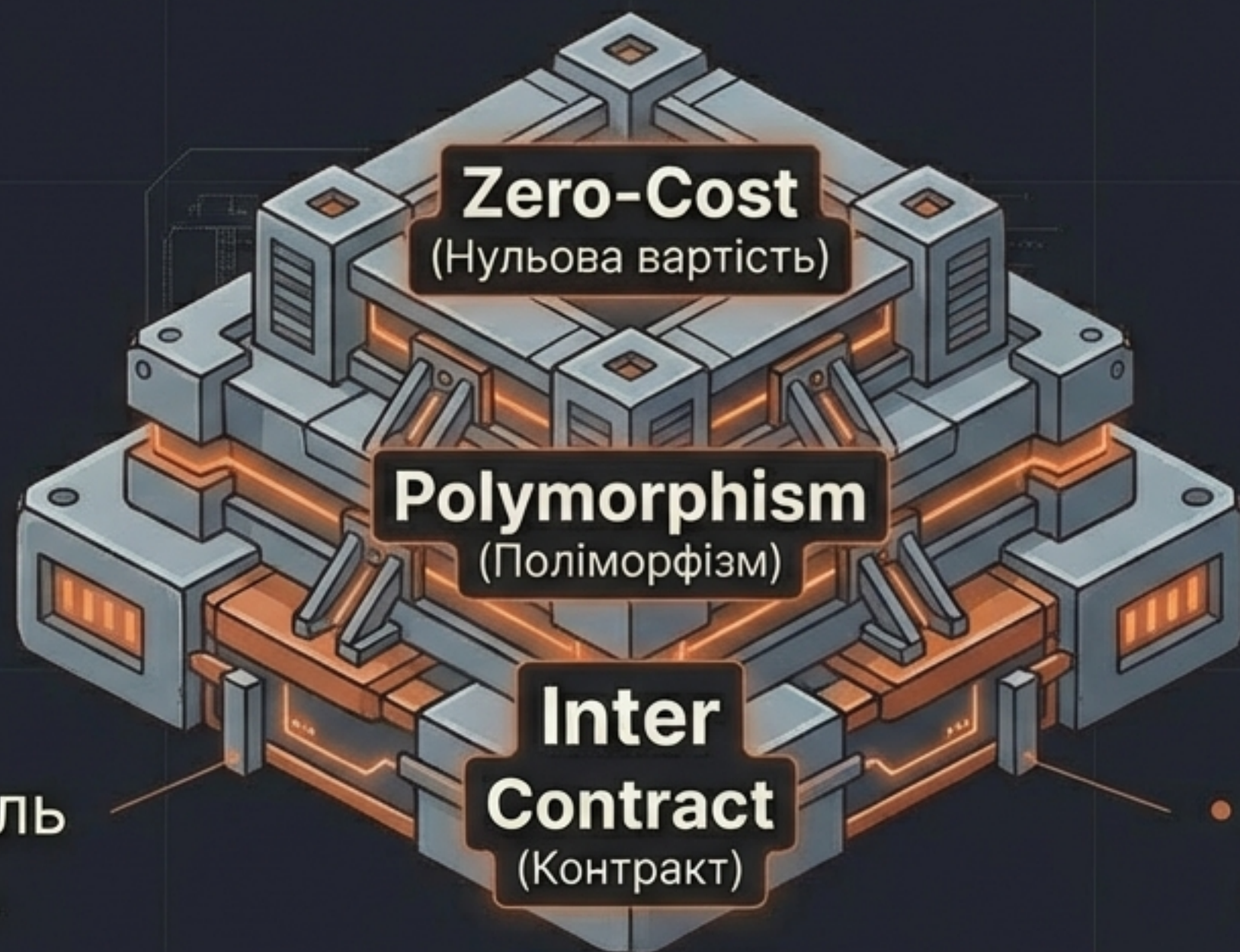
```
impl Debug for Point {  
    x: i32,  
    y: i32,  
}
```

```
impl PartialEq for Point {  
    x: i32,  
    y: i32,  
}
```

```
impl Clone for Point {  
    ...  
}
```

Метапрограмування: Генерація коду компілятором.

ПІДСУМОК: Трейт як фундамент Rust



Inter

- Системна модель узгодженості.

Inter

- Композиція > Спадкування.

Inter

- Надійність + Швидкість.